

Biobank and Biospecimen Management System

Database Project Report

Course: CBIO204 – Fundamentals of Databases

Student Name: Mohamed Rateb

Student ID: 231002599

Date: August 2026

1. Introduction

This project presents a relational database system for managing a biobank and its biological samples. The main purpose of the system is to organize information about donors, biological samples, storage locations, researchers, sample requests, sample usage, and laboratory test requests.

The database was designed using MySQL. It includes multiple related tables, primary and foreign keys, integrity constraints, views, and a trigger to support important business rules. The system also includes sample data and SQL queries that demonstrate how the database can be used in a real laboratory or research environment.

The database contains 11 main tables and was designed to reduce data duplication and keep the stored information consistent and organized.

2. Problem Statement and Project Scope

2.1 Problem Statement

Biobanks manage a large amount of information about donors, biological samples, storage locations, researchers, and laboratory activities. Managing this information manually can make it difficult to track samples, identify their donors, know where samples are stored, and monitor how samples are requested and used.

The purpose of this project is to create a database that organizes these activities in one system. The database allows users to store donor information, record sample collection events, manage different sample types, track sample storage and aliquots, and record research requests and laboratory tests.

2.2 Project Scope

The system covers the main activities involved in managing biological samples. These include:

- Managing donor information.
- Recording donor consent.
- Recording sample collection events.
- Managing different sample types.
- Tracking biological samples.
- Managing storage locations.
- Tracking aliquots and their storage locations.
- Managing researchers.
- Recording sample requests.
- Recording sample usage.
- Managing laboratory test requests.

The system focuses on database management and does not include a separate graphical user interface.

2.3 Assumptions

The following assumptions were made during the database design:

1. Each donor has a unique donor code.
2. Each biological sample belongs to one collection event.
3. Each sample has one defined sample type.
4. Each aliquot belongs to one sample and one storage location.
5. Researchers can submit requests for samples.
6. A sample can be requested by different researchers.
7. Sample usage is recorded through the sample request and aliquot information.
8. Storage positions should be unique to avoid assigning two locations to the same physical position.
9. Sample and aliquot volumes must be greater than zero.

10. Only anonymized donor identifiers are used in the database.

3. Entities and Attributes

The database contains 11 main entities.

3.1 Donor

The Donor table stores basic information about each donor.

Main attributes:

- donor_id – Primary Key.
- donor_code – Unique identifier for the donor.
- date_of_birth – Donor date of birth.
- sex – Donor sex.
- blood_group – Blood group.
- registration_date – Date when the donor was registered.

3.2 Consent

The Consent table stores information about donor consent.

Main attributes:

- consent_id – Primary Key.
- donor_id – Foreign Key referencing Donor.
- consent_date.
- consent_type.
- status.
- expiry_date.

3.3 Collection_Event

The Collection_Event table records when and how biological material was collected from a donor.

Main attributes:

- collection_id – Primary Key.
- donor_id – Foreign Key referencing Donor.
- collection_date.
- collection_method.
- collected_by.
- notes.

3.4 Sample_Type

The Sample_Type table stores the different types of biological samples.

Examples include blood, plasma, serum, tissue, DNA, RNA, saliva, urine, bone marrow, and CSF.

Main attributes:

- sample_type_id – Primary Key.
- type_name – Unique sample type name.
- description.

3.5 Sample

The Sample table stores information about biological samples.

Main attributes:

- sample_id – Primary Key.
- collection_id – Foreign Key referencing Collection_Event.
- sample_type_id – Foreign Key referencing Sample_Type.
- sample_code – Unique sample identifier.
- collection_date.
- volume_ml.
- sample_status.

A check constraint is used to make sure that the sample volume is greater than zero.

3.6 Storage_Location

The Storage_Location table records where samples or aliquots are stored.

Main attributes:

- location_id – Primary Key.
- freezer_code.
- rack_code.
- box_code.
- position_code.
- temperature_c.

The combination of freezer, rack, box, and position is unique.

3.7 Aliquot

The Aliquot table stores smaller portions of biological samples.

Main attributes:

- aliquot_id – Primary Key.
- sample_id – Foreign Key referencing Sample.
- aliquot_code – Unique aliquot identifier.
- volume_ml.
- created_date.
- status.
- location_id – Foreign Key referencing Storage_Location.

3.8 Researcher

The Researcher table stores information about researchers who request or use samples.

Main attributes:

- researcher_id – Primary Key.
- researcher_code – Unique researcher identifier.
- researcher_name.
- email – Unique email address.
- department.

- institution.

3.9 Sample_Request

The Sample_Request table records requests made by researchers for biological samples.

Main attributes:

- request_id – Primary Key.
- researcher_id – Foreign Key referencing Researcher.
- sample_id – Foreign Key referencing Sample.
- request_date.
- purpose.
- status.

3.10 Sample_Usage

The Sample_Usage table records the actual use of aliquots.

Main attributes:

- usage_id – Primary Key.
- request_id – Foreign Key referencing Sample_Request.
- aliquot_id – Foreign Key referencing Aliquot.
- usage_date.
- volume_used_ml.
- usage_purpose.

3.11 Test_Request

The Test_Request table records laboratory tests requested for samples.

Main attributes:

- test_request_id – Primary Key.
- sample_id – Foreign Key referencing Sample.
- researcher_id – Foreign Key referencing Researcher.
- test_type.

- request_date.
- status.
- result.

4. Entity Relationship Diagram

The Entity Relationship Diagram was created using MySQL Workbench. It shows the 11 entities in the database and the relationships between them.

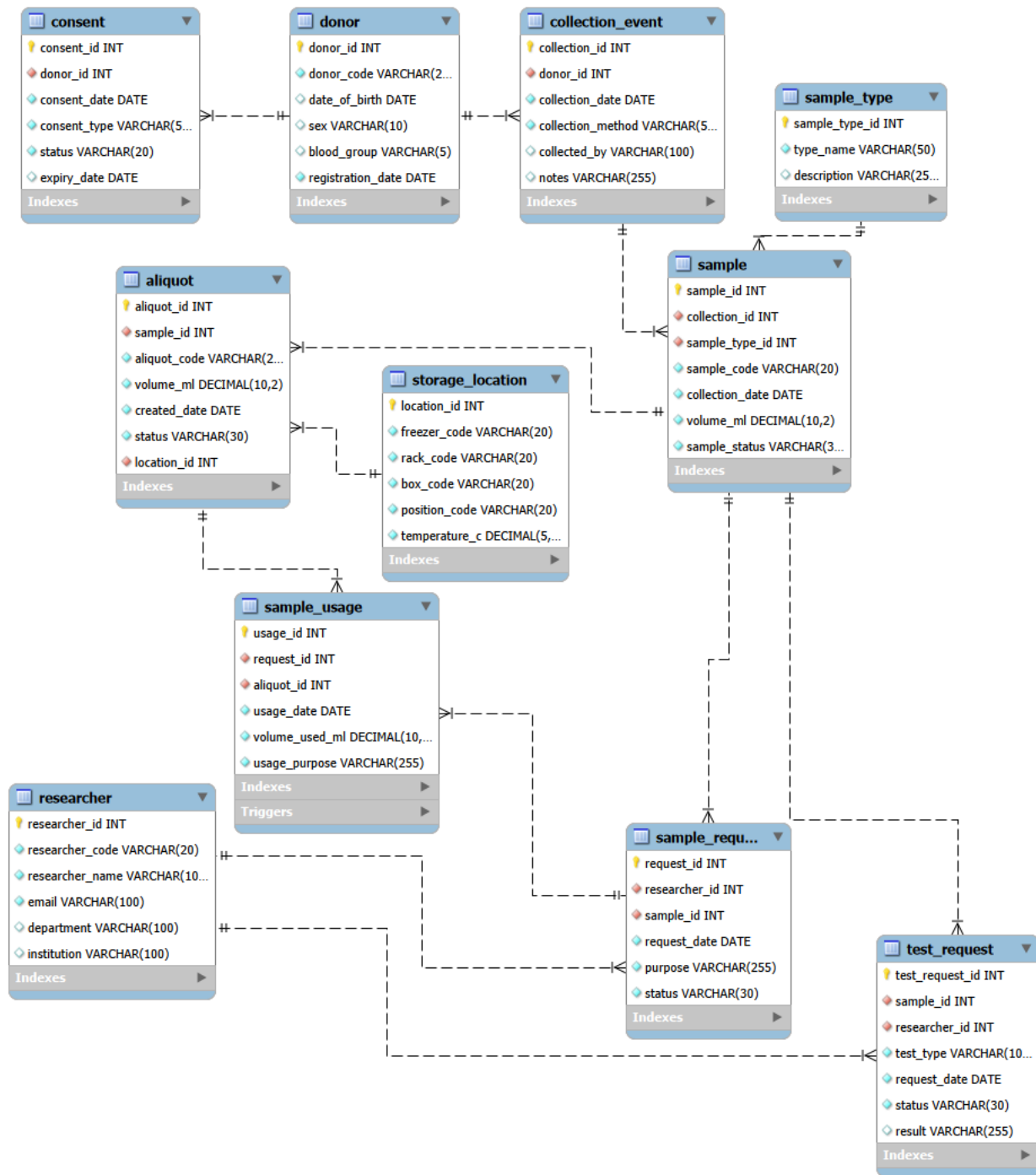


Figure 1. Entity Relationship Diagram of the Biobank Database.

The main relationships include:

- A donor can have multiple consent records.
- A donor can have multiple collection events.

- A collection event can be associated with samples.
- A sample belongs to one sample type.
- A sample can have multiple aliquots.
- An aliquot is stored in a storage location.
- A researcher can submit multiple sample requests.
- A sample can be requested by different researchers through Sample_Request.
- Sample usage connects requests with aliquots.
- Researchers can also submit test requests for samples.

The ER diagram also shows the primary keys and foreign keys used to connect the tables.

5. Relational Schema and Constraints

The database was implemented as a relational schema using MySQL. Each main entity was converted into a table with a primary key. Foreign keys were used to connect related tables.

Primary keys uniquely identify records in each table. Foreign keys maintain relationships between tables and help preserve referential integrity.

Several additional constraints were also used. For example, donor codes, sample codes, aliquot codes, and researcher codes are unique. Researcher emails are also unique.

Check constraints were used for values such as sample volume, aliquot volume, and usage volume to make sure they are greater than zero.

The storage location table also uses a unique constraint on the combination of freezer, rack, box, and position. This prevents the same physical storage position from being assigned more than once.

5.1 Formal Relational Schema (Notation)

Each entity is mapped to a relation as follows, with primary keys (PK) and foreign keys (FK) shown for every table:

```
Donor(donor_id PK, donor_code, date_of_birth, sex, blood_group, registration_date)
Consent(consent_id PK, donor_id FK->Donor, consent_date, consent_type, status,
expiry_date)
Collection_Event(collection_id PK, donor_id FK->Donor, collection_date,
collection_method,
```

```

    collected_by, notes)
Sample_Type(sample_type_id PK, type_name, description)
Sample(sample_id PK, collection_id FK->Collection_Event, sample_type_id FK->Sample_Type,
        sample_code, collection_date, volume_ml, sample_status)
Storage_Location(location_id PK, freezer_code, rack_code, box_code, position_code,
temperature_c)
Aliquot(aliquot_id PK, sample_id FK->Sample, aliquot_code, volume_ml, created_date,
status,
        location_id FK->Storage_Location)
Researcher(researcher_id PK, researcher_code, researcher_name, email, department,
institution)
Sample_Request(request_id PK, researcher_id FK->Researcher, sample_id FK->Sample,
request_date, purpose, status)
Sample_Usage(usage_id PK, request_id FK->Sample_Request, aliquot_id FK->Aliquot,
usage_date, volume_used_ml, usage_purpose)
Test_Request(test_request_id PK, sample_id FK->Sample, researcher_id FK->Researcher,
test_type, request_date, status, result)

```

6. Database Normalization

The database was normalized up to Third Normal Form (3NF).

6.1 First Normal Form (1NF)

The tables satisfy 1NF because each column contains a single atomic value and there are no repeating groups. Each table also has a primary key that uniquely identifies its records.

For example, the Donor table stores one value for each donor attribute such as date of birth, sex, and blood group.

6.2 Second Normal Form (2NF)

The database satisfies 2NF because the tables are already in 1NF and their non-key attributes depend on the complete primary key.

Most tables use a single-column primary key, such as donor_id, sample_id, and researcher_id. Therefore, there are no partial dependencies on part of a composite key.

6.3 Third Normal Form (3NF)

The database satisfies 3NF by separating information that has its own independent dependencies into different tables.

For example, sample type information is stored in the Sample_Type table instead of being repeated in every Sample record. The Sample table only stores sample_type_id as a foreign key.

Similarly, storage information is stored in Storage_Location, while Aliquot stores only the location_id foreign key.

Researcher information is also stored separately in the Researcher table. The Sample_Request table stores only researcher_id rather than repeating the researcher's name, email, department, and institution.

This design reduces data duplication and makes the database easier to maintain and update.

7. Database Implementation

The database was implemented using MySQL. The implementation was divided into separate SQL scripts to keep the project organized.

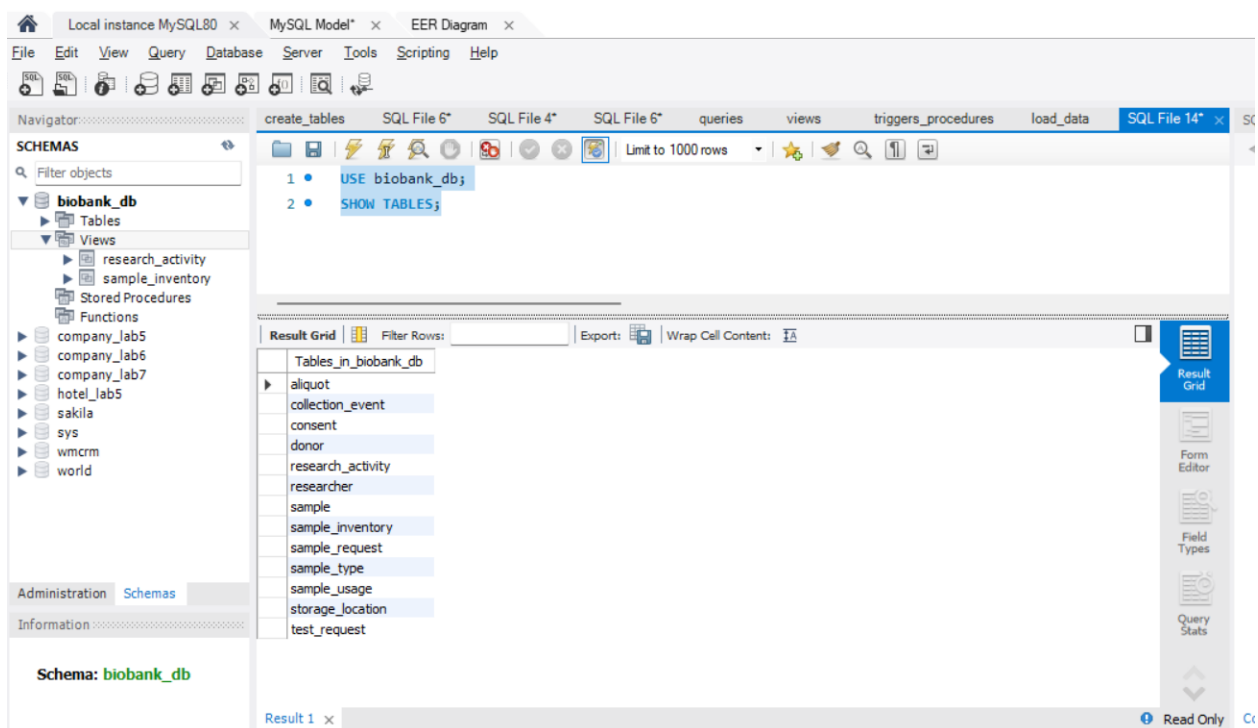


Figure 2. Database tables successfully created in MySQL Workbench.

The main SQL files are:

- create_tables.sql – creates the database tables and constraints.
- load_data.sql – inserts sample data.
- queries.sql – contains retrieval, join, aggregation, subquery, insert, update, and delete operations.
- views.sql – contains the database views.

- triggers_procedures.sql – contains the trigger used in the project.

The following excerpts from create_tables.sql show two representative tables, including primary keys, foreign keys, and constraints:

```
3 • USE biobank_db;
4 • CREATE TABLE Donor (
5     donor_id INT AUTO_INCREMENT PRIMARY KEY,
6     donor_code VARCHAR(20) NOT NULL UNIQUE,
7     date_of_birth DATE,
8     sex VARCHAR(10),
9     blood_group VARCHAR(5),
10    registration_date DATE NOT NULL
11 );
```

Figure 3. CREATE TABLE definition for the Donor table.

```

41 • CREATE TABLE Sample (
42     sample_id INT AUTO_INCREMENT PRIMARY KEY,
43     collection_id INT NOT NULL,
44     sample_type_id INT NOT NULL,
45     sample_code VARCHAR(20) NOT NULL UNIQUE,
46     collection_date DATE NOT NULL,
47     volume_ml DECIMAL(10,2) NOT NULL,
48     sample_status VARCHAR(30) NOT NULL,
49
50     CONSTRAINT fk_sample_collection
51         FOREIGN KEY (collection_id)
52         REFERENCES Collection_Event(collection_id),
53
54     CONSTRAINT fk_sample_type
55         FOREIGN KEY (sample_type_id)
56         REFERENCES Sample_Type(sample_type_id),
57
58     CONSTRAINT chk_sample_volume
59         CHECK (volume_ml > 0)
60 );
61 • CREATE TABLE Storage_Location (

```

Output

Action Output			
#	Time	Action	Message
66	12:48:56	SELECT * FROM Research_Activity LIMIT 0, 1000	10 row(s) returned

Figure 4. CREATE TABLE definition for the Sample table, including constraints.

The database contains 11 tables and uses primary keys, foreign keys, unique constraints, check constraints, and a trigger to enforce important database rules.

8. Sample Data and SQL Operations

Test data was added to demonstrate that the database works with realistic records.

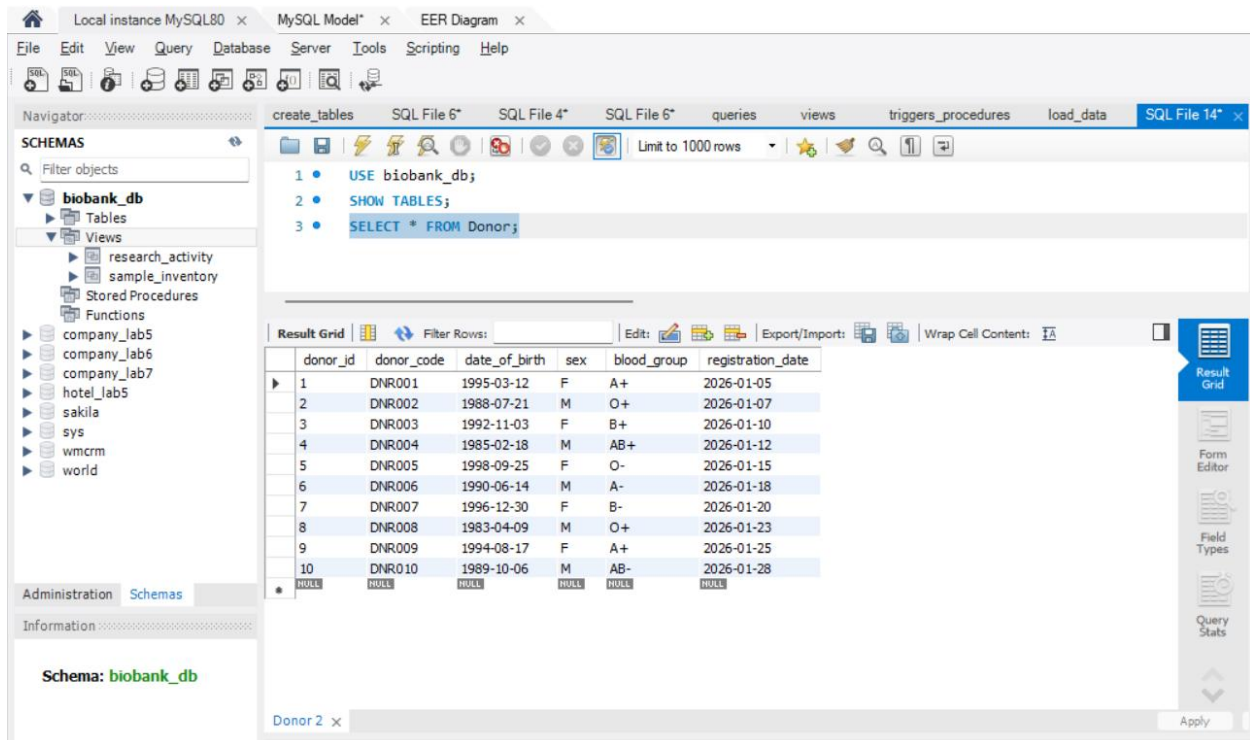


Figure 5. Sample donor records successfully retrieved from the database.

The database also supports SQL joins to combine information from multiple related tables. The following example retrieves sample information together with donor and sample type information.

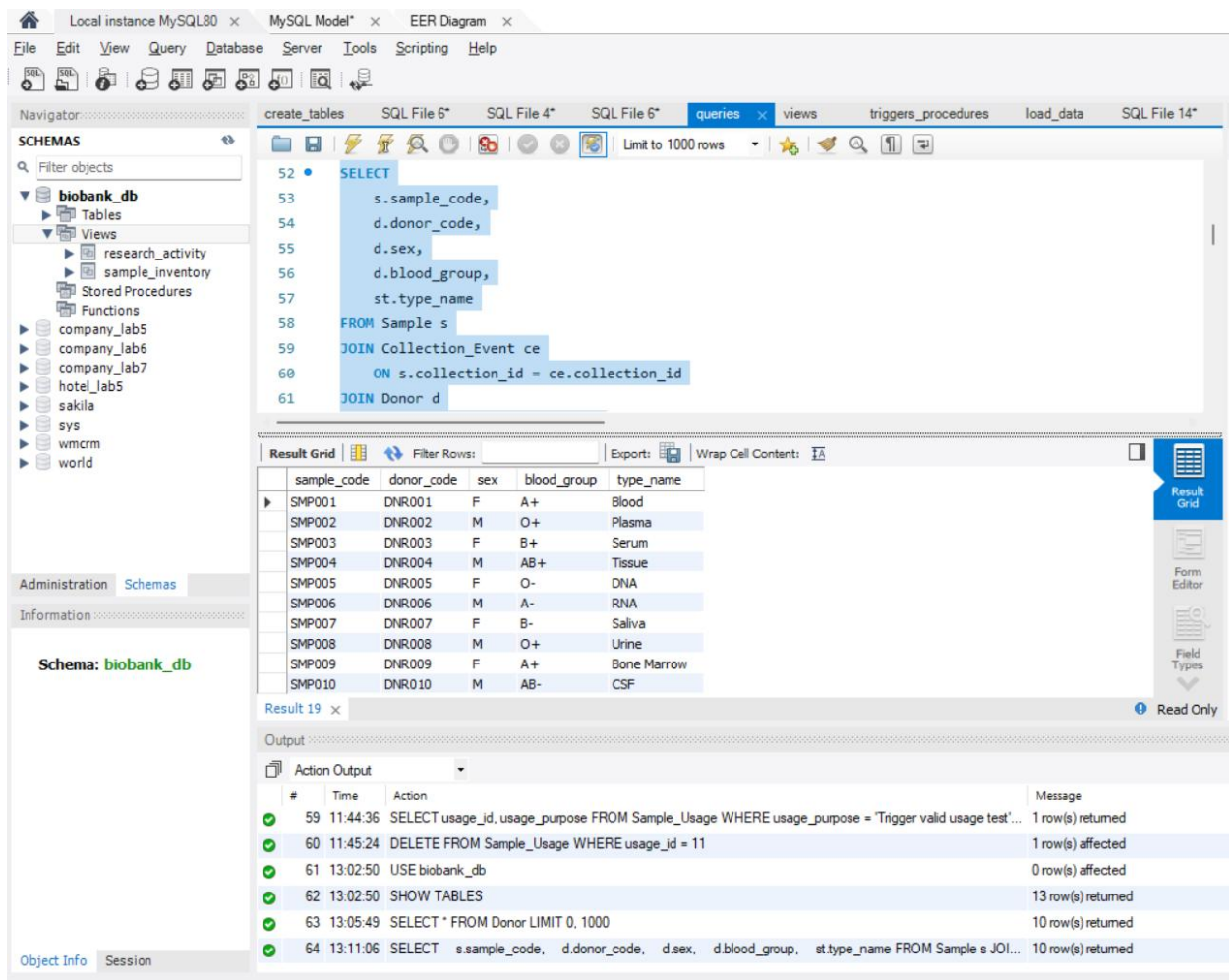


Figure 6. Successful execution of a multi-table JOIN query.

The main tables contain meaningful sample records representing donors, researchers, samples, sample types, storage locations, requests, sample usage, and test requests.

The database also includes different types of SQL operations.

Basic Retrieval

Examples include displaying all donors, female donors, donors with a specific blood group, and donors born after a specific date.

Joins

Joins were used to combine information from related tables. For example, sample information can be displayed together with the donor and sample type information.

Aggregation

Aggregation queries were used to count donors by blood group, count samples by sample type, and calculate total and average sample volume.

Subqueries

Subqueries were used to find samples with a volume greater than the average and researchers who have submitted approved requests.

Data Modification

The project also includes INSERT, UPDATE, and DELETE operations to demonstrate database modification.

9. Views and Trigger

9.1 Views

Two views were created to simplify access to frequently used information.

The first view, `sample_inventory`, provides information related to samples and their storage.

The screenshot displays the MySQL Workbench interface. On the left, the Navigator pane shows the 'biobank_db' schema with various objects including 'research_activity' and 'sample_inventory' views. The main editor window shows the SQL script for creating and querying these views. The 'Result Grid' pane displays the output of the 'sample_inventory' view, showing a table with 9 columns: sample_code, sample_type, sample_volume_ml, sample_status, aliquot_code, aliquot_volume_ml, aliquot_status, freezer_code, and r. The 'Action Output' pane at the bottom shows the execution log, indicating that the queries were executed successfully and returned the expected number of rows.

sample_code	sample_type	sample_volume_ml	sample_status	aliquot_code	aliquot_volume_ml	aliquot_status	freezer_code	r
SMP001	Blood	10.00	Available	ALQ001	2.00	Available	FZ01	R
SMP002	Plasma	8.00	Available	ALQ002	2.00	Available	FZ01	R
SMP003	Serum	7.50	Available	ALQ003	1.50	Available	FZ01	R
SMP004	Tissue	15.00	Available	ALQ004	3.00	Available	FZ01	R
SMP005	DNA	5.00	Available	ALQ005	1.00	Available	FZ01	R
SMP006	RNA	4.00	Available	ALQ006	1.00	Available	FZ01	R
SMP007	Saliva	12.00	Available	ALQ007	2.50	Available	FZ01	R
SMP008	Urine	20.00	Available	ALQ008	4.00	Available	FZ01	R
SMP009	Bone Marrow	6.00	Available	ALQ009	1.50	Available	FZ02	R

Figure 7. Successful execution of the sample_inventory view.

The second view, research_activity, provides information about researchers and their sample requests and activities.

The screenshot displays the MySQL Workbench interface. The Navigator pane on the left shows the 'biobank_db' schema with tables like 'research_activity' and 'sample_inventory'. The central SQL editor shows the definition of the 'research_activity' view, which is a SELECT statement joining 'sample_inventory' and 'research_activity' tables. The Result Grid below the editor displays the data returned by the view, including columns for researcher_name, department, sample_code, request_date, request_purpose, request_status, aliquot_code, and usage. The Action Output pane at the bottom shows the execution log, indicating that the view was successfully created and executed, returning 10 rows.

researcher_name	department	sample_code	request_date	request_purpose	request_status	aliquot_code	usage
Ahmad Khalil	Genetics	SMP001	2026-02-05	Genetic analysis	Completed	ALQ001	2026-01-01
Sara Nasser	Oncology	SMP002	2026-02-06	Cancer biomarker study	Approved	ALQ002	2026-01-01
Omar Haddad	Molecular Biology	SMP003	2026-02-07	Molecular analysis	Approved	ALQ003	2026-01-01
Lina Saad	Immunology	SMP004	2026-02-08	Immunology research	Approved	ALQ004	2026-01-01
Yousef Mansour	Genomics	SMP005	2026-02-09	Genomic sequencing	Approved	ALQ005	2026-01-01
Maya Saleh	Bioinformatics	SMP006	2026-02-10	Bioinformatics study	Approved	ALQ006	2026-01-01
Khaled Omar	Cell Biology	SMP007	2026-02-11	Cell biology research	Approved	ALQ007	2026-01-01
Rania Fares	Cancer Biology	SMP008	2026-02-12	Cancer research	Approved	ALQ008	2026-01-01
Tariq Amin	Pathology	SMP009	2026-02-13	Pathology analysis	Approved	ALQ009	2026-01-01

Figure 8. Successful execution of the research_activity view.

Views make it easier to retrieve useful information without repeatedly writing complex joins.

9.2 Trigger

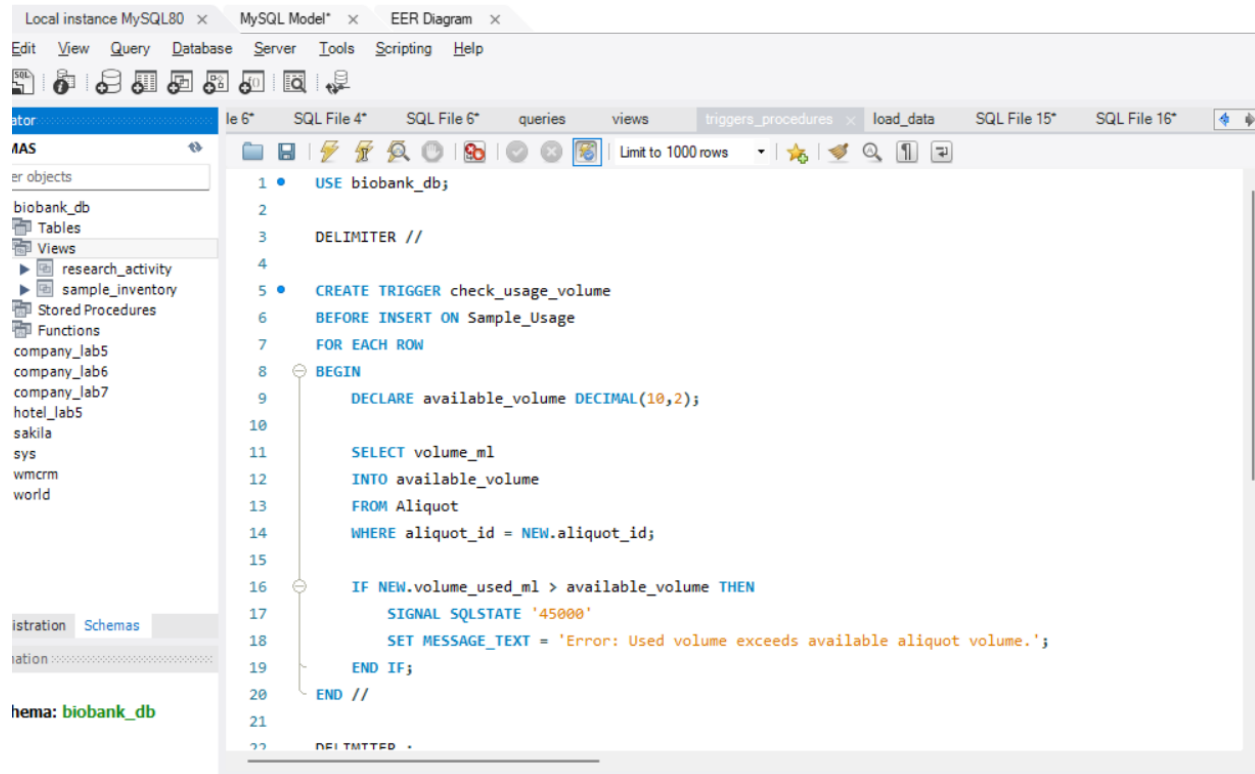
A trigger was created to prevent the use of more aliquot volume than is available.

The trigger checks the requested volume_used_ml before a new record is inserted into Sample_Usage.

If the requested volume is greater than the available aliquot volume, the database rejects the operation and returns an error.

This represents an important business rule because the system should not allow researchers to use more biological material than is available.

The following screenshot shows the trigger definition executed successfully in MySQL Workbench:



The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'Schemas' tab with a tree view of databases including 'biobank_db'. The main editor window shows a SQL script for creating a trigger. The script is as follows:

```
1 • USE biobank_db;
2
3 DELIMITER //
4
5 • CREATE TRIGGER check_usage_volume
6 BEFORE INSERT ON Sample_Usage
7 FOR EACH ROW
8 BEGIN
9     DECLARE available_volume DECIMAL(10,2);
10
11     SELECT volume_ml
12     INTO available_volume
13     FROM Aliquot
14     WHERE aliquot_id = NEW.aliquot_id;
15
16     IF NEW.volume_used_ml > available_volume THEN
17         SIGNAL SQLSTATE '45000'
18         SET MESSAGE_TEXT = 'Error: Used volume exceeds available aliquot volume.';
19     END IF;
20 END //
21
22 DELIMITER ;
```

Figure 9. Trigger definition (*check_usage_volume*) created successfully in MySQL Workbench.

10. Testing and Results

The trigger was tested using two different cases. First, a valid usage of 0.50 mL was inserted successfully. Then, an invalid usage of 5.00 mL was attempted, which exceeded the available aliquot volume. The trigger correctly rejected the operation and returned an error.

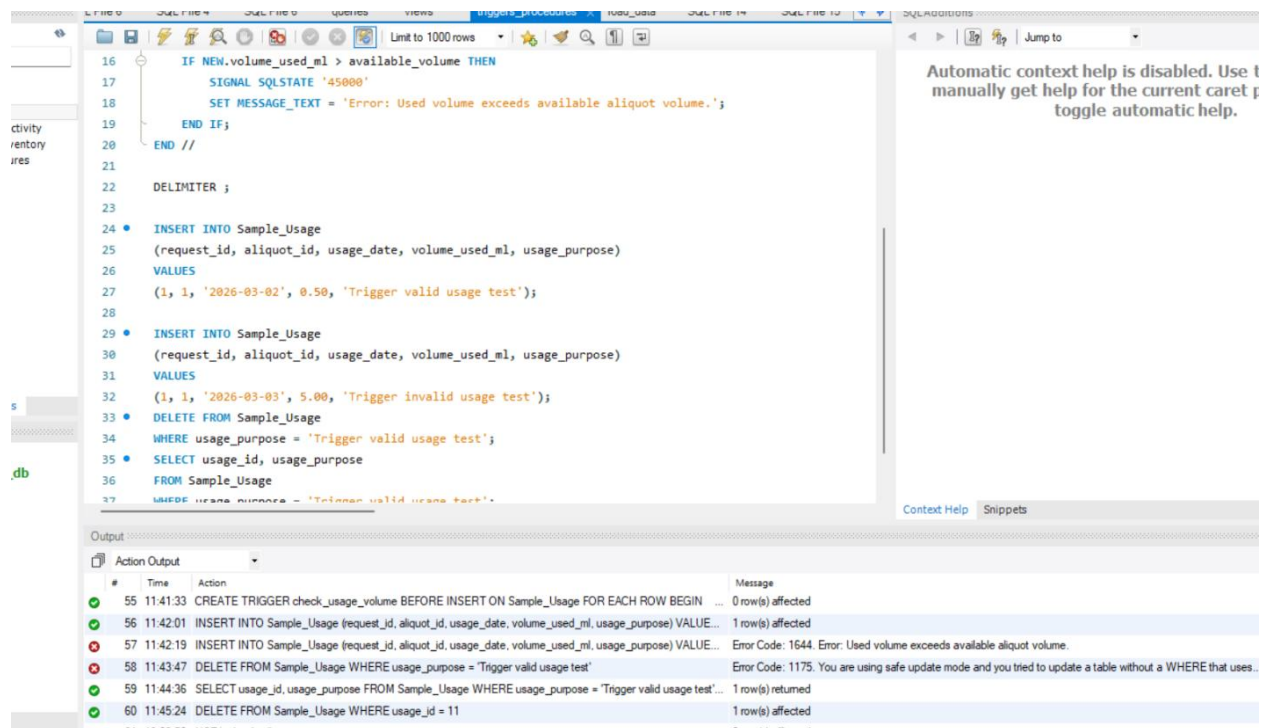


Figure 10. Trigger implementation and testing with valid and invalid sample usage cases.

Several tests were performed to make sure that the database and its business rules work correctly.

A valid sample usage operation was first tested using 0.50 mL. The operation was successfully inserted into the database.

A second test attempted to use 5.00 mL from an aliquot that contained only 2.00 mL.

The database rejected the operation and returned the error:

"Used volume exceeds available aliquot volume."

This confirmed that the trigger works correctly and prevents invalid sample usage.

Other database operations, including queries, joins, aggregation, subqueries, views, inserts, updates, and deletes, were also executed during testing.

11. Limitations and Conclusion

11.1 Limitations

The current system focuses mainly on database management and does not include a graphical user interface. Users interact with the database through SQL and MySQL Workbench.

The database also uses sample data for demonstration purposes rather than real donor or clinical information.

11.2 Conclusion

This project successfully designed and implemented a relational database for managing a biobank and biological samples.

The system contains 11 related tables and uses primary keys, foreign keys, unique constraints, check constraints, views, and a trigger. The database was normalized to 3NF to reduce unnecessary duplication and improve data consistency.

The SQL queries and testing demonstrate that the database can support common biobank activities such as managing donors, tracking samples, handling storage information, processing researcher requests, recording sample usage, and managing laboratory test requests.

Overall, the project demonstrates the complete database development process, from identifying requirements and designing the ER model to implementing, querying, and testing the database in MySQL.

12. References

Oracle. (2026). *MySQL 8.0 Reference Manual: Creating and Using a Database*.

<https://dev.mysql.com/doc/refman/8.0/en/database-use.html>

Oracle. (2026). *MySQL 8.0 Reference Manual: Creating a Table*.

<https://dev.mysql.com/doc/refman/8.0/en/creating-tables.html>

Oracle. (2026). *MySQL 8.0 Reference Manual: FOREIGN KEY Constraints*.

<https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

Oracle. (2026). *MySQL 8.0 Reference Manual: View Syntax*.

<https://dev.mysql.com/doc/refman/8.0/en/view-syntax.html>

Oracle. (2026). *MySQL 8.0 Reference Manual: Using Views*.

<https://dev.mysql.com/doc/refman/8.0/en/views.html>

Oracle. (2026). *MySQL 8.0 Reference Manual: Trigger Syntax and Examples*.

<https://dev.mysql.com/doc/refman/8.0/en/trigger-syntax.html>

Oracle. (2026). *MySQL 8.0 Reference Manual: Using Foreign Keys*.

<https://dev.mysql.com/doc/refman/8.0/en/example-foreign-keys.html>